

JAR

Swarna Sankalp

Goal-based savings and adaptive pacing for Jar

Jar already made saving gold easy. The harder question is what someone is saving for, how much they can keep up, and what happens the month they cannot.

7

product layers

100

synthetic users

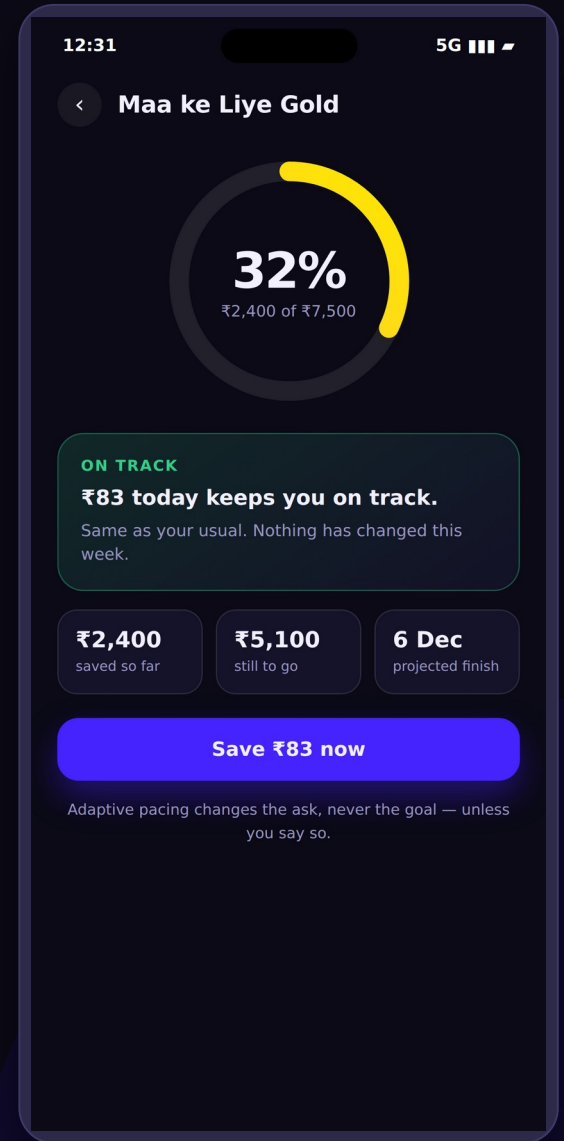
21

model features

4

models compared

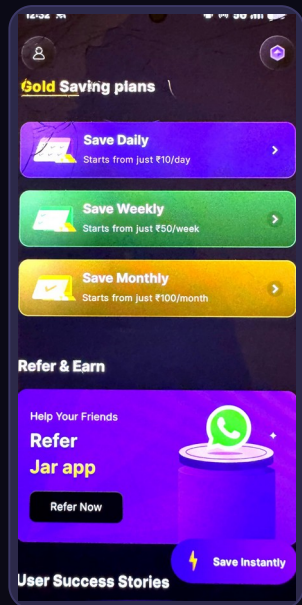
PRD · ML prototype · interactive UI



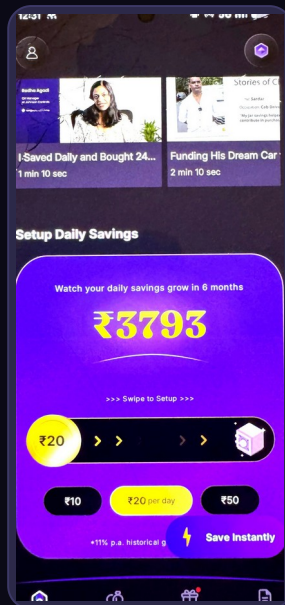
Research

What Jar already does well

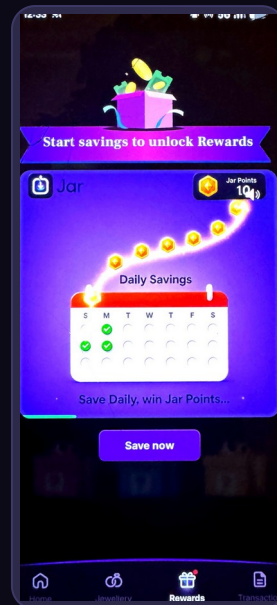
Taken from the live app. Jar has solved access: multiple cadences, a projection of what daily saving becomes, and a rewards loop that brings people back. None of it asks what the money is for.



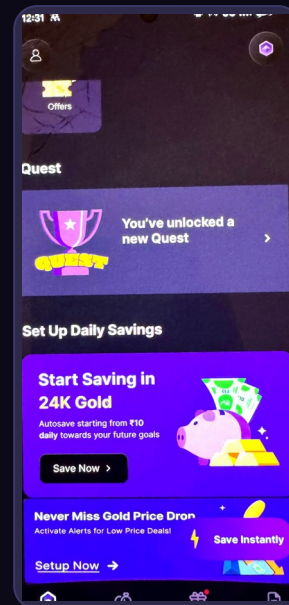
Daily, weekly and monthly cadences already exist



₹20/day projected to ₹3,793 in six months



Saving streaks feed a points and rewards loop



Quests and autosave nudges on the home feed

So the gap is not saving

Every screen answers "how do I save?" and none answers "what am I saving for, and how much can I actually keep up?"

Swarna Sankalp adds that layer. It is not a new savings product — it sits on top of the cadences already shipped.

The problem

Three questions a saver cannot answer alone

People understand that saving is good. They stall on the specifics.

01

Why should I save now?

Saving with no destination is abstract and repetitive. There is nothing to be loyal to.

02

How much should I save?

A number picked at random is either too small to matter or too big to keep up.

03

What if I cannot this month?

A missed contribution currently reads as failure. Most people quietly stop after that.

Swarna Sankalp answers all three: **a goal** for the first, **a computed rate** for the second, **adaptive pacing and recovery** for the third.

Seven layers, one journey

Each layer answers a question the user is actually asking at that moment.

1 Motivation

"What am I saving for?"

Cultural and personal goal templates, plus custom goals.

2 Planning

"How much do I need?"

Target amount and date become a required saving rate.

3 Recommendation

"How much should I save?"

A contribution and cadence that fit past behaviour.

4 Adaptive pacing

"Can I afford that today?"

The ask flexes with observed capacity. The goal does not.

5 Recovery

"I fell behind, now what?"

Increase, extend, reduce or pause. Never a failure state.

6 Progress

"Am I on track?"

Progress, projected completion, light nudges.

7 Redemption

"What happens when I reach it?"

Keep saving, digital gold, coin, or jewellery. User picks.

Layers 4 and 5, in gold, are the actual innovation.
The rest is table stakes that makes them usable.

From an occasion to a number you can keep up

A goal template is the hook. What matters is the line under the number that explains where it came from, because an unexplained recommendation gets ignored.

Templates, not a blank form

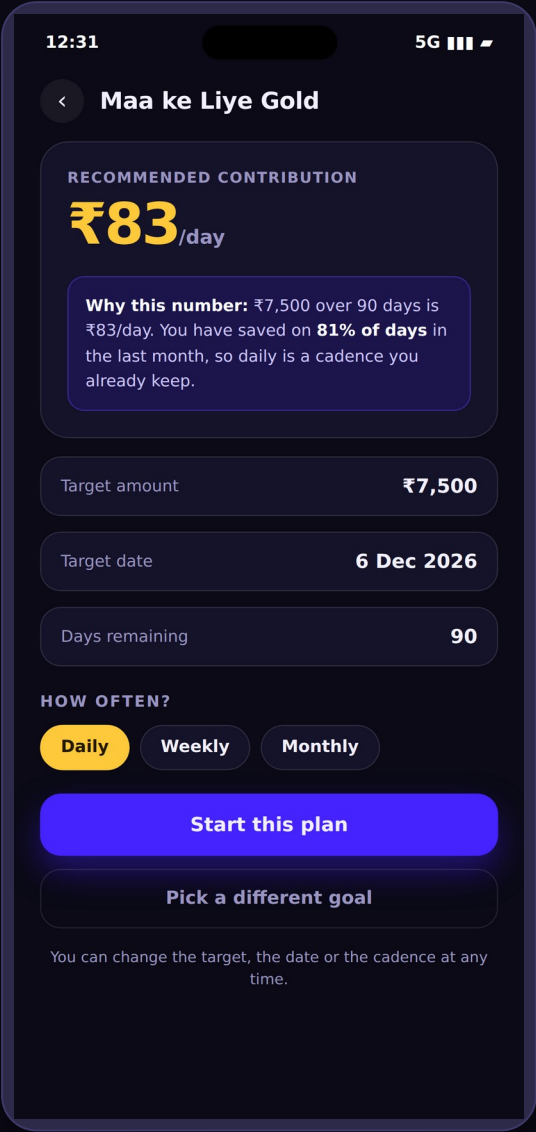
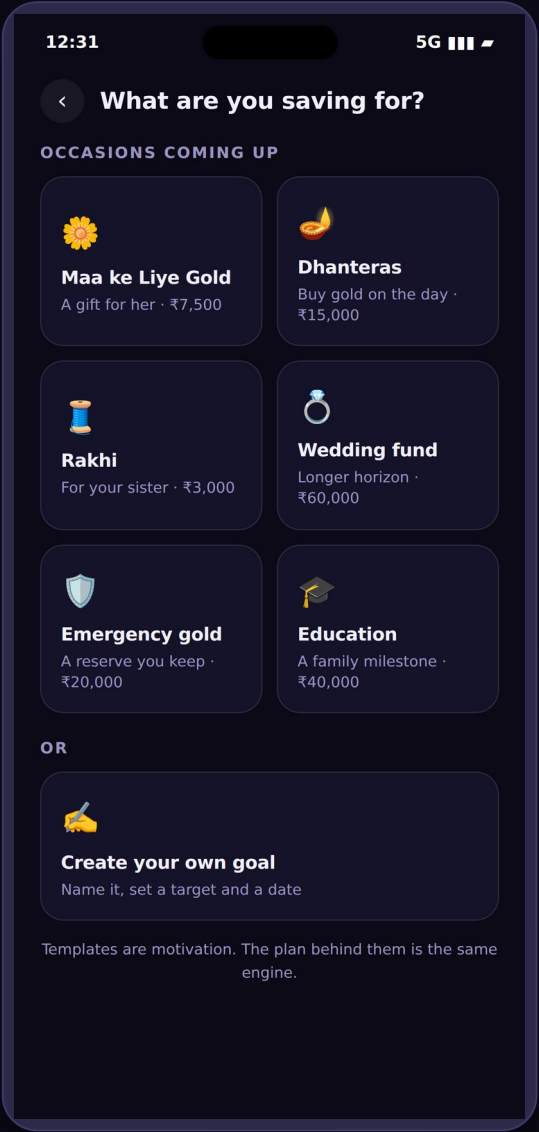
Maa ke Liye Gold, Dhanteras, Rakhi, wedding, emergency gold. A blank amount field is where most goal features die.

The rate is arithmetic, not magic

₹7,500 over 90 days is ₹83 a day. Shown plainly so nobody has to trust a black box.

Cadence follows behaviour

Daily is only recommended because this user already saved on 81% of days last month.



The ask moves. The goal stays put.

This is the whole idea. When capacity drops, the contribution drops with it and the shortfall shifts to later weeks. The target and the date are untouched unless the user changes them.

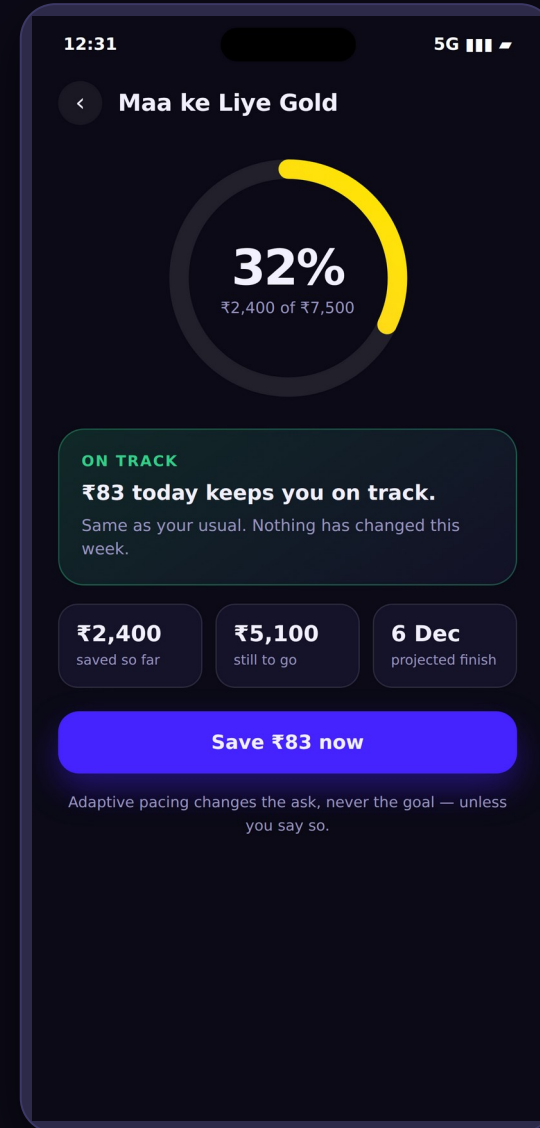
Normal week

"₹83 today keeps you on track."

Tighter week

"₹42 today is enough."

Contributions were smaller for five days, so the ask halves. The button halves with it — asking for ₹42 and then showing a ₹83 button is how trust in a feature like this dies.



Missing a week is normal. The product should say so.

The default behaviour of most savings apps here is a red streak and silence. That is the moment users quit. Recovery turns it into a choice with four honest answers.

Save a little more

₹83 → ₹90/day

Spread the shortfall over the days left.
Same finish date.

Move the date

finish 13 Dec instead

Keep the daily amount, accept a later finish.

Lower the target

₹7,500 → ₹6,900

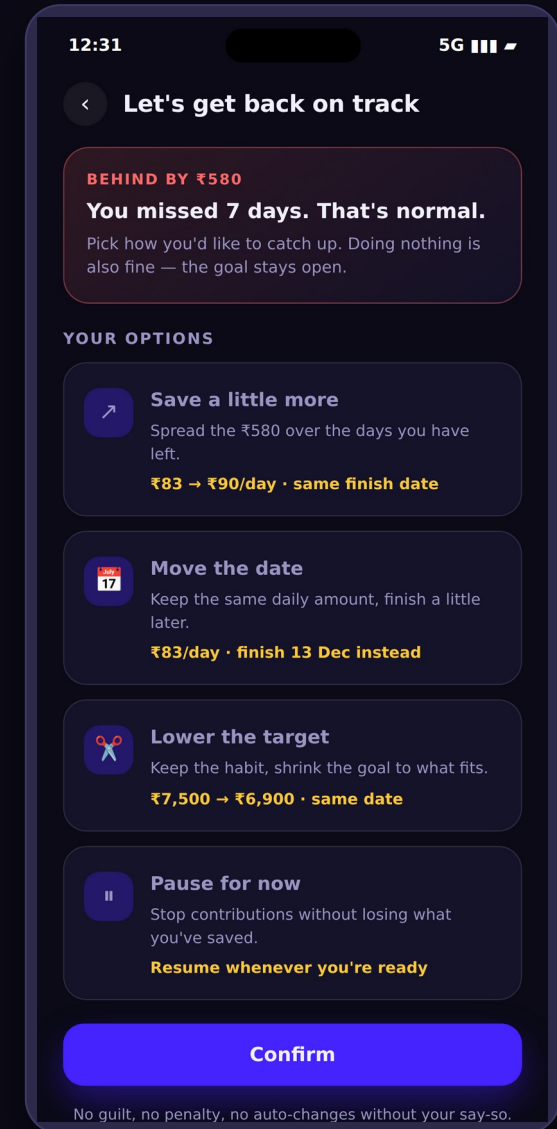
Keep the habit, shrink the goal to what fits.

Pause for now

nothing is lost

Stop contributing without losing what is already saved.

Doing nothing is also a valid answer. The goal stays open and nothing is auto-changed.



Reaching the goal is not a sales trigger

This is where a savings product usually turns into a funnel. Four options are offered, one of them is "keep saving", and none is pre-selected.

Keep saving

Roll straight into a new goal. A perfectly good ending.

Digital gold

Hold the balance as 24K and do nothing else.

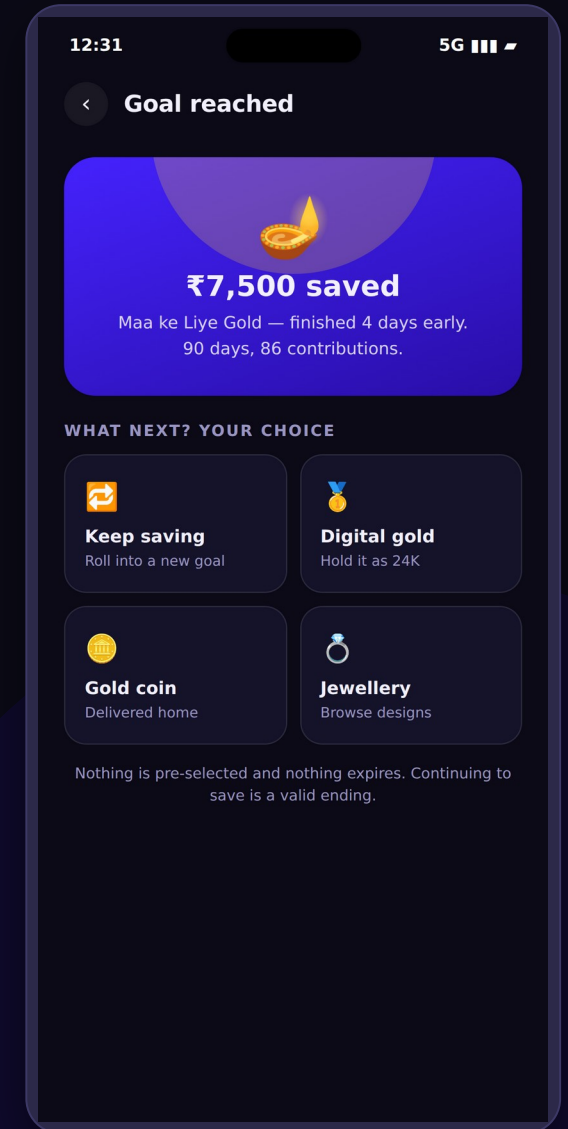
Gold coin

Physical delivery, if that was the point of the goal.

Jewellery

Browse designs, only if the goal implied it.

The PRD deliberately does not hard-code an 85% completion trigger. Optimal timing is a thing to test, not to assert.



The intelligence layer

A working model behind the recommendation

10,441 synthetic transactions across 100 users over six months, aggregated to monthly features, then trained on the earlier months and tested on the later ones.



21

features per user-month

400/100

train / test rows, split by time

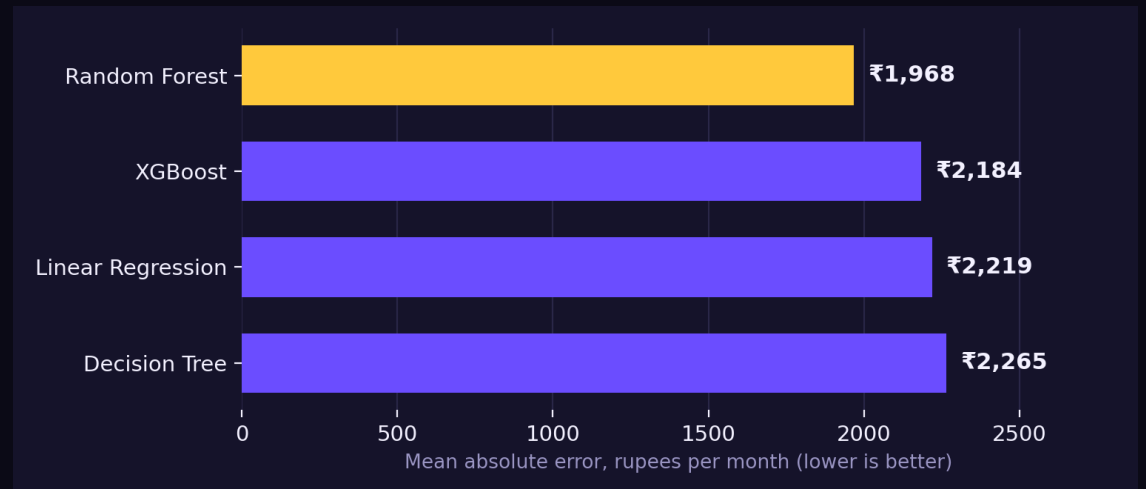
₹1,968

best mean absolute error

0.88

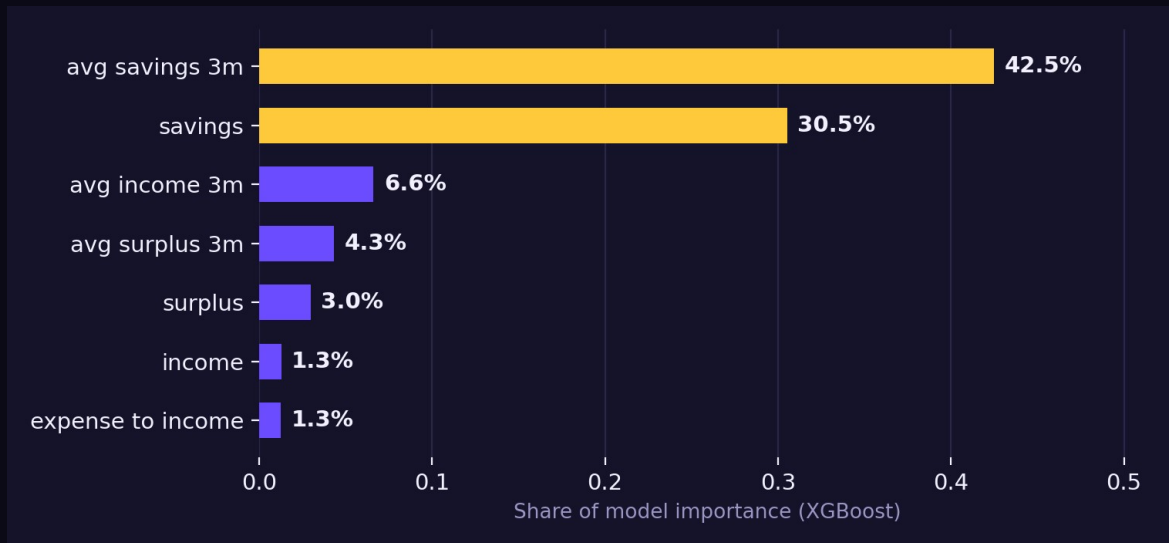
R² on held-out months

Random Forest wins on every metric, but the notebook ships XGBoost as the final model. Worth switching, or justifying.



What the model actually learned

And one problem worth fixing before this ships



Past saving behaviour dominates. Income barely registers. That is reassuring — it means the model is learning habit, which is what the PRD says to optimise for.

The recommendations are too aggressive

Across the test set, the median user currently saves ₹4,068 a month and the engine recommends ₹15,876. Every single test user — 100% of them — is told to save more.

Why it happens

The target is 80% of next month's surplus, and the only safety rule caps the answer at current surplus. Both are ceilings on what is arithmetically possible, not on what someone will actually sustain.

The fix

Cap any single step-up as a multiple of what the user already saves — say 1.3× — and let the recommendation climb over months. The PRD already names this risk: "optimise for adherence and sustainability, not maximum daily contribution."

How success is judged

One north star, and a guardrail that can veto it

NORTH STAR

Successful Goal Progress Rate

Share of active goal users who stay on track — not completions, not deposits.

Goal activation rate

Do people start one at all?

Saving adherence

Actual contribution vs recommended.

D30 / D60 / D90 retention

Are goals still alive months later?

Incremental savings

Treatment minus control. The causal test.

The guardrail

Missed-contribution rate among users on an adaptive plan.

If a feature designed to make saving sustainable ends up increasing the number of missed contributions, it has failed — no matter how good activation and completion look. That single metric can veto a launch.

What gets tested, not asserted

Goal-led onboarding vs generic entry · adaptive vs static contribution · which recovery option works · redemption timing.

What I would build, and what I would ask

Three versions, and the things I cannot know

V1

Goal layer

Templates, target and date, required rate, cadence recommendation, progress. Runs on Jar-native data only.

V2

Adaptive and recovery

Affordability signal, plan switch, recovery engine. Adds user-declared income cycle.

V3

Intelligent and commerce

Stronger modelling, consented external context where appropriate, redemption paths.

Open questions for Jar

- Is there already an internal goal engine or goal metadata layer?
- Which savings products can actually be attached to a goal today?
- What user-level savings data is available to product models?
- Which segments show the best contribution sustainability now?

Written from the public Jar app and public strategy. No access to Jar data, analytics or roadmap. Every number in the model section comes from synthetic data.